MIDTERM LAB ACTIVITY - 5

Learning outcomes

- Create nodes and connect them using Python object references.
- Implement a singly linked list with forward traversal.
- Implement a doubly linked list with forward and backward navigation.
- Handle empty lists, single nodes, and boundary conditions correctly.

Activity scenario

You are developing two small applications for a campus student center. The first manages customers waiting for service. The second lets students move forward and backward through a music playlist. Complete both applications using linked lists.

General instructions

- Use Python 3 and create your own Node and linked list classes.
- Store records in linked nodes. Do not use a Python list, deque, or dictionary as a substitute for either linked list.
- You may use input(), loops, strings, and temporary variables. A node reference such as next points to another object; it is not a numeric memory address.
- Submit two runnable .py files, output screenshots, and answers to the reflection questions.
- Use the starter code as a guide. Replace every TODO or pass with your implementation.

Required files

StudentID_Surname_SinglyLinkedList.py
StudentID_Surname_DoublyLinkedList.py
StudentID_Surname_Reflection.pdf or .docx

Part A Campus service waiting list

One way linked list | 20 points

Customers join the end of a waiting list. Staff serve the customer at the front. A customer may also cancel their place. Each node stores a customer name and a next reference.

Required operations

- add_customer(name): Add a customer at the tail. Maintain both head and tail references.
- serve_customer(): Remove and return the head customer. If the list is empty, display a clear message.
- cancel_customer(name): Remove the first matching customer. Update head or tail when needed; report a missing name.
- display_waiting(): Traverse from head to tail and display all names in order.
- count_customers(): Count customers by traversing the nodes.

Starter code

```
class CustomerNode:
    def __init__(self, name):
        self.name = name
        self.next = None

class WaitingList:
    def __init__(self):
        self.head = None
        self.tail = None

    def add_customer(self, name):
        pass  # TODO

    def serve_customer(self):
        pass  # TODO

    def cancel_customer(self, name):
        pass  # TODO

    def display_waiting(self):
        pass  # TODO

    def count_customers(self):
        pass  # TODO

# TODO: Create a menu and call the appropriate methods.
```

Menu and demonstration

Menu: 1 Add customer | 2 Serve customer | 3 Cancel customer | 4 Display waiting list | 5 Count customers | 0 Exit

Add Ana, Ben, and Carla in that order. Serve Ana, cancel Carla, and add Daniel. Your final display must match the order below. Display formatting may differ.

```
Waiting: Ana -> Ben -> Carla -> None
Served: Ana
Cancelled: Carla
Waiting: Ben -> Daniel -> None
Customers waiting: 2
```

Part B Campus music playlist

Two way linked list | 20 points

Each song node stores a title, prev, and next. Keep head, tail, and current references. Use a linear playlist: the first song has no previous song and the last song has no next song.

Required operations

- add_song(title): Append a song. Set current to the first song when adding to an empty playlist.
- next_song() and previous_song(): Move current when a neighboring song exists. At a boundary, display a message and keep current unchanged.
- show_current(): Display the current song, or report that the playlist is empty.
- display_forward() and display_backward(): Traverse from head using next and from tail using prev.
- remove_song(title): Remove the first matching song and reconnect both links. If it is current, select its next song, otherwise its previous song. If none remains, set current to None.

Starter code

```
class SongNode:
    def __init__(self, title):
        self.title = title
        self.prev = None
        self.next = None

class Playlist:
    def __init__(self):
        self.head = self.tail = self.current = None

    # TODO: Implement these methods:
    # add_song(self, title)
    # next_song(self)
    # previous_song(self)
    # show_current(self)
    # display_forward(self)
    # display_backward(self)
    # remove_song(self, title)

# TODO: Create an interactive menu for all operations.
```

Demonstration

Add Sunrise, Ocean, and Starlight. Show current, move next, move previous, then remove Ocean. Display the playlist in both directions.

```
Current: Sunrise
After next: Ocean
After previous: Sunrise
Removed: Ocean
Forward: Sunrise <-> Starlight
Backward: Starlight <-> Sunrise
```

Reference rules to check

When nonempty: head.prev is None and tail.next is None. For adjacent nodes A and B, A.next is B and B.prev is A. When empty, head, tail, and current must all be None.

Testing and assessment

Required test cases

- Both applications: Attempt operations on an empty structure. Confirm that the program does not crash.
- Waiting list: Add one customer, then serve them. Confirm both head and tail become None.
- Waiting list: Cancel the head, a middle node, the tail, and a name that does not exist.
- Playlist: Try previous at the first song and next at the last song. Confirm current stays unchanged.
- Playlist: Remove the head, a middle song, the tail, and the current song. Check forward and backward order after each removal.
- Playlist: Remove the only song. Confirm all three references become None.
- Both applications: Add duplicate names or titles and show that cancellation or removal affects only the first match.
- Both menus: Enter an invalid option and return to the menu without crashing.

Reflection questions

1. Why is a singly linked list suitable for the customer waiting list?

2. How does the prev reference help the music playlist?

3. When removing a middle song, which references must be updated, and why?

4. State the time complexity of adding a customer with a tail reference, serving a customer, cancelling by name, moving to the next song, and displaying all songs. Explain each answer.

5. Describe one bug you found during testing and how you fixed it.

Marking rubric

Area	Criteria	Points
Singly linked list	Correct node structure, head and tail management	4
Singly linked list	Add and serve operations	6
Singly linked list	Cancel first matching customer, including boundaries	6
Singly linked list	Forward display and traversal based count	4
Doubly linked list	Node structure and correct bidirectional references	4
Doubly linked list	Append, navigation, current song and boundary behavior	6
Doubly linked list	Removal and current song selection	6
Doubly linked list	Forward and backward display	4
Testing and usability	Required tests, readable menus and invalid input handling	6
Reflection	Accurate explanations and complexity analysis	4

Total: 50 points